Published as a conference paper at ICLR 2026

# TRINITY: AN EVOLVED LLM COORDINATOR

**Jinglue Xu**[1*], **Qi Sun**[1,3*], **Peter Schwendeman**[2†],
**Stefan Nielsen**[1], **Edoardo Cetin**[1], **Yujin Tang**[1]
[1] Sakana AI, Japan [2] University of Michigan, USA [3] Institute of Science Tokyo, Japan

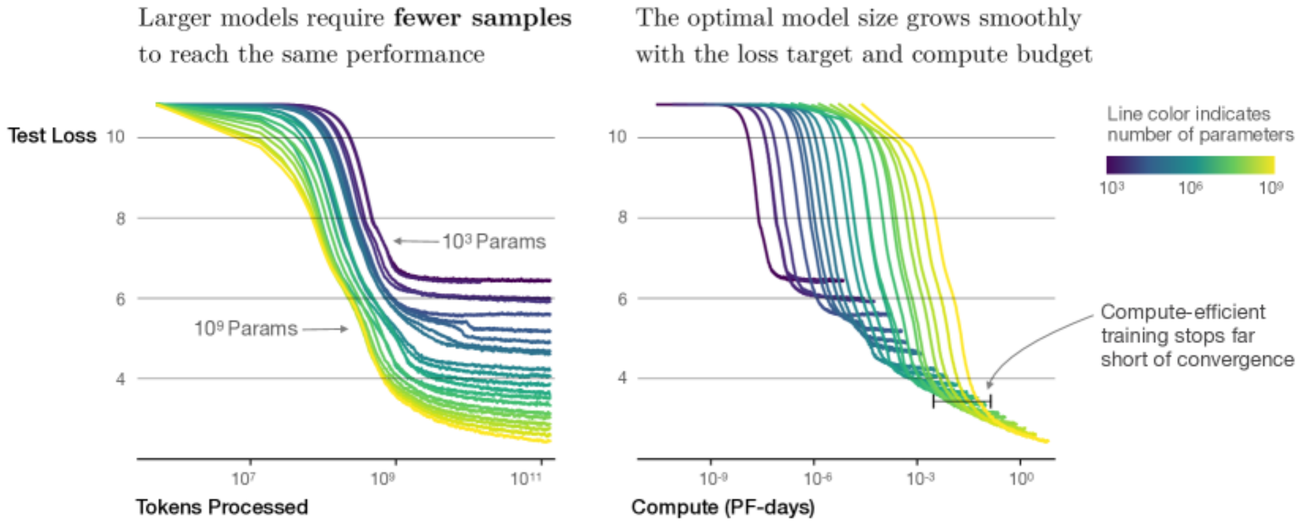
Presenter: Yifei Li, 2026/07/06

# How to improve LLM's performance?

## Motivation

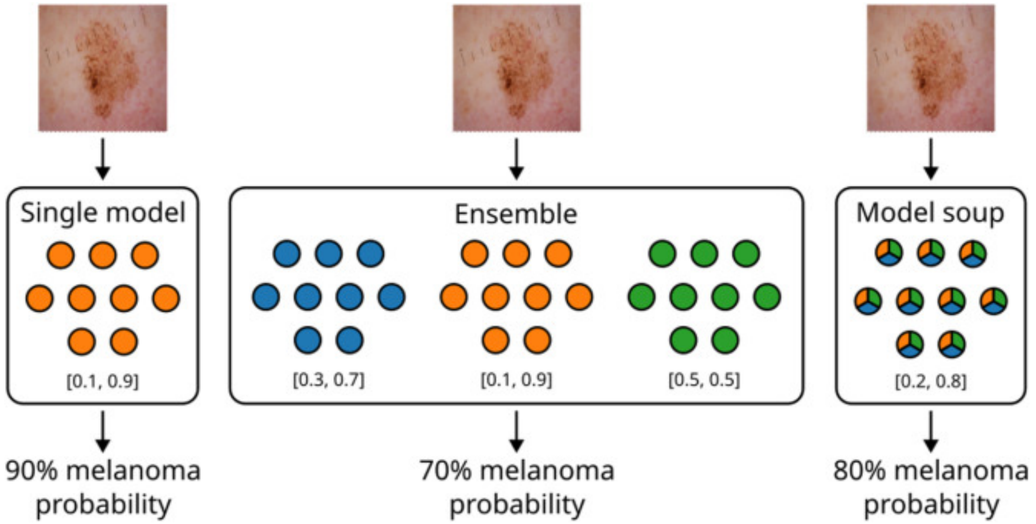
### Scaling LLMs

bigger models, more tokens, more compute




### Model merging at "micro-level"

combine weights of several LLMs into one



# Test-time model composition

## At a “macro-level”

- Trinity fuses the complementary strengths of diverse, state-of-the-art LLMs — without modifying LLMs’ weight.

# Problem formulation

## Coordinator is a policy


*state $s_i$ = query + turns so far (captured by a penultimate token hidden vec) · action $a_i$ = (agent, role) · $\pi_\theta$ = coordinator*

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s\_T) \qquad \text{with horizon} \quad T \le B\_turn$$

$\tau$ is one complete rollout (the full multi-turn coordination episode). B_turn caps how many turns the loop is allowed — the process also stops early the moment a Verifier turn ACCEPTs.

# Problem formulation

## The objective: maximize expected terminal reward

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)] \qquad \theta^* = \operatorname{argmax}_\theta J(\theta)$$

Notice: Cost here is the number of multi-turn rollout during training

### Terminal & binary

$R(\tau) \in \{0, 1\}$ is only observed at the very end of the rollout. No intermediate per-turn signal.

### ≈ 10,000-dim θ

lightweight head parameters and singular-value scale parameters.

### Costly samples

Every single evaluation of J(θ) means running a multi-turn rollout through real LLMs. A budget B_env caps how many you can afford.

# Method

## Tri-role coordination

### Thinker

Devises high-level strategy: plans, decomposes the problem, or critiques a partial solution.

### Worker

Executes: produces concrete progress — a derivation, code, or a numeric result.

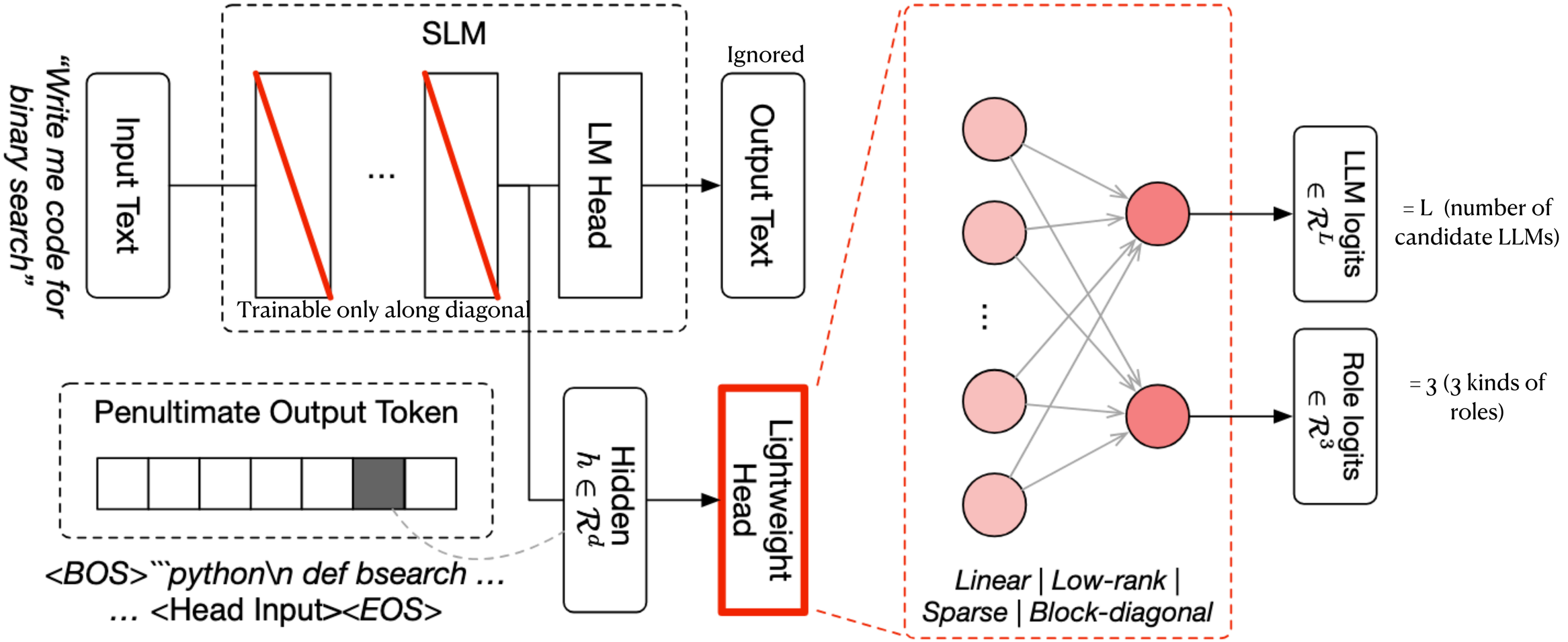
### Verifier

Judges the accumulated solution: ACCEPT (done) or REVISE (continue), with a diagnosis.

**Stopping rule:** the process halts the moment a Verifier turn ACCEPTs the current answer — otherwise it continues until the K-turn budget (K=5 by default) is exhausted.

# Method

## Efficient parametrization



# Evolutionary Strategy

## Problem Statement

- Task: **minimize** an **objective function** (*fitness* function, *loss* function) in continuous domain

$$f : \mathcal{X} \subseteq \mathbb{R}^n \to \mathbb{R}, \qquad \boldsymbol{x} \mapsto f(\boldsymbol{x})$$

- **Black Box** scenario (direct search scenario)

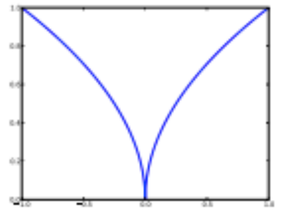

  - gradients are not available or not useful
  - problem domain specific knowledge is used only within the black box, e.g. within an appropriate encoding
- Search costs: number of function evaluations

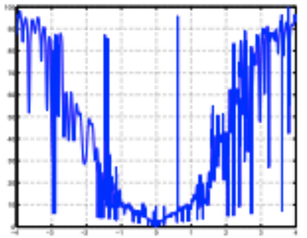
Screenshot: https://www.cmap.polytechnique.fr/~nikolaus.hansen/gecco2013-CMA-ES-tutorial.pdf

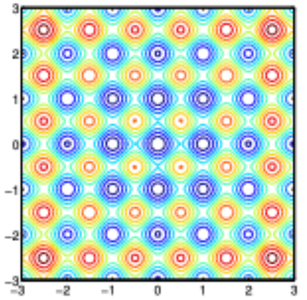
# Evolutionary Strategy

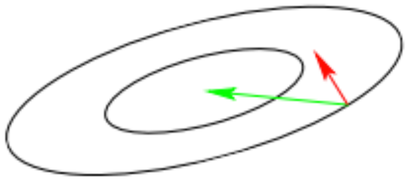
## Why stochastic search?

- non-linear, non-quadratic, non-convex
  on linear and quadratic functions much better search policies are available
- ruggedness
  non-smooth, discontinuous, multimodal, and/or noisy function
- dimensionality (size of search space)
  (considerably) larger than three
- non-separability
  dependencies between the objective variables
- ill-conditioning







gradient direction Newton direction

Screenshot: https://www.cmap.polytechnique.fr/~nikolaus.hansen/gecco2013-CMA-ES-tutorial.pdf

# Covariance matrix adaptation evolution strategy (CMA-ES)

## Algorithm

**Intuition: Increasing the sample probability of successful candidate solutions or search directions is beneficial.**

**set** $\lambda$ // number of samples per iteration, at least two, generally > 4
**initialize** $m$, $\sigma$, $C = I$, $p_\sigma = 0$, $p_c = 0$ // initialize state variables
**while** *not terminate* **do** // iterate
    **for** $i$ **in** $\{1 \dots \lambda\}$ **do** // sample $\lambda$ new solutions and evaluate them
        $x_i = \text{sample\_multivariate\_normal}(\text{mean} = m, \text{ covariance\_matrix} = \sigma^2 C)$
        $f_i = \text{fitness}(x_i)$
    $x_{1 \dots \lambda} \leftarrow x_{s(1) \dots s(\lambda)}$ with $s(i) = \text{argsort}(f_{1 \dots \lambda}, i)$ // sort solutions
    $m' = m$ // we need later $m - m'$ and $x_i - m'$
    $m \leftarrow \text{update\_m}(x_1, \dots, x_\lambda)$ // move mean to better solutions
    $p_\sigma \leftarrow \text{update\_ps}(p_\sigma, \sigma^{-1} C^{-1/2} (m - m'))$ // update isotropic evolution path
    $p_c \leftarrow \text{update\_pc}(p_c, \sigma^{-1} (m - m'), \|p_\sigma\|)$ // update anisotropic evolution path
    $C \leftarrow \text{update\_C}(C, p_c, (x_1 - m')/\sigma, \dots, (x_\lambda - m')/\sigma)$ // update covariance matrix
    $\sigma \leftarrow \text{update\_sigma}(\sigma, \|p_\sigma\|)$ // update step-size using isotropic path length
**return** $m$ or $x_1$

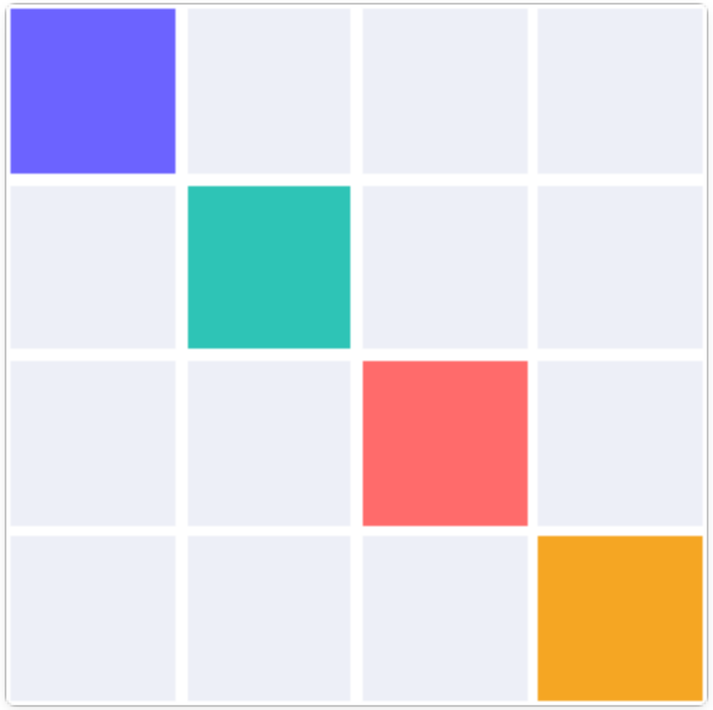
Why not using this: computationally infeasible to keep the full nxn covariance matrix when n could be 10k.

Make it a diagonal matrix!

Screenshot: https://en.wikipedia.org/wiki/CMA-ES#Algorithm

# Is the diagonal trade-off free?

A block-diagonal-10 head — forcing near-total independence between logits — keeps most of the performance despite ≈10× fewer weights than the default linear head.



```
block-ε-separability:  cross-block coupling is O(ε), ε small
```

**When we can use a diagonal matrix**: a diagonal covariance can track per-block step sizes almost as well as a full one would, because the blocks genuinely don't interact much (proved in Appendix A.1 + empirical result)

*Once block-ε-separability holds, the "trade-off" from full → diagonal covariance stops being a real accuracy loss but a pure efficiency win.*

# Concrete setting for sep-CMA-ES

$$n \approx 10{,}000 \qquad \lambda = \lfloor 4 + 3 \cdot \ln n \rfloor = 32 \qquad m_{CMA} = 16 \text{ rollouts/candidate}$$

$$T = \lfloor B_{env} / (m_{CMA} \cdot \lambda) \rfloor = \lfloor B_{env} / 512 \rfloor \quad \text{iterations}$$

Cost for one iteration

1. Sample $\lambda=32$ candidate head-vectors $\theta_i$ from $N(m, \mathrm{diag}(\sigma^2))$
2. Evaluate each candidate over $m_{CMA}=16$ real multi-turn rollouts, average the binary rewards to get a denoised fitness score
3. Rank candidates; update mean m toward the fitness-weighted top performers
4. Update each per-parameter $\sigma_i$ independently, using the evolution path
5. Repeat for $T = \lfloor B_{env}/512 \rfloor$ generations — the number of iterations the fixed atomic budget affords

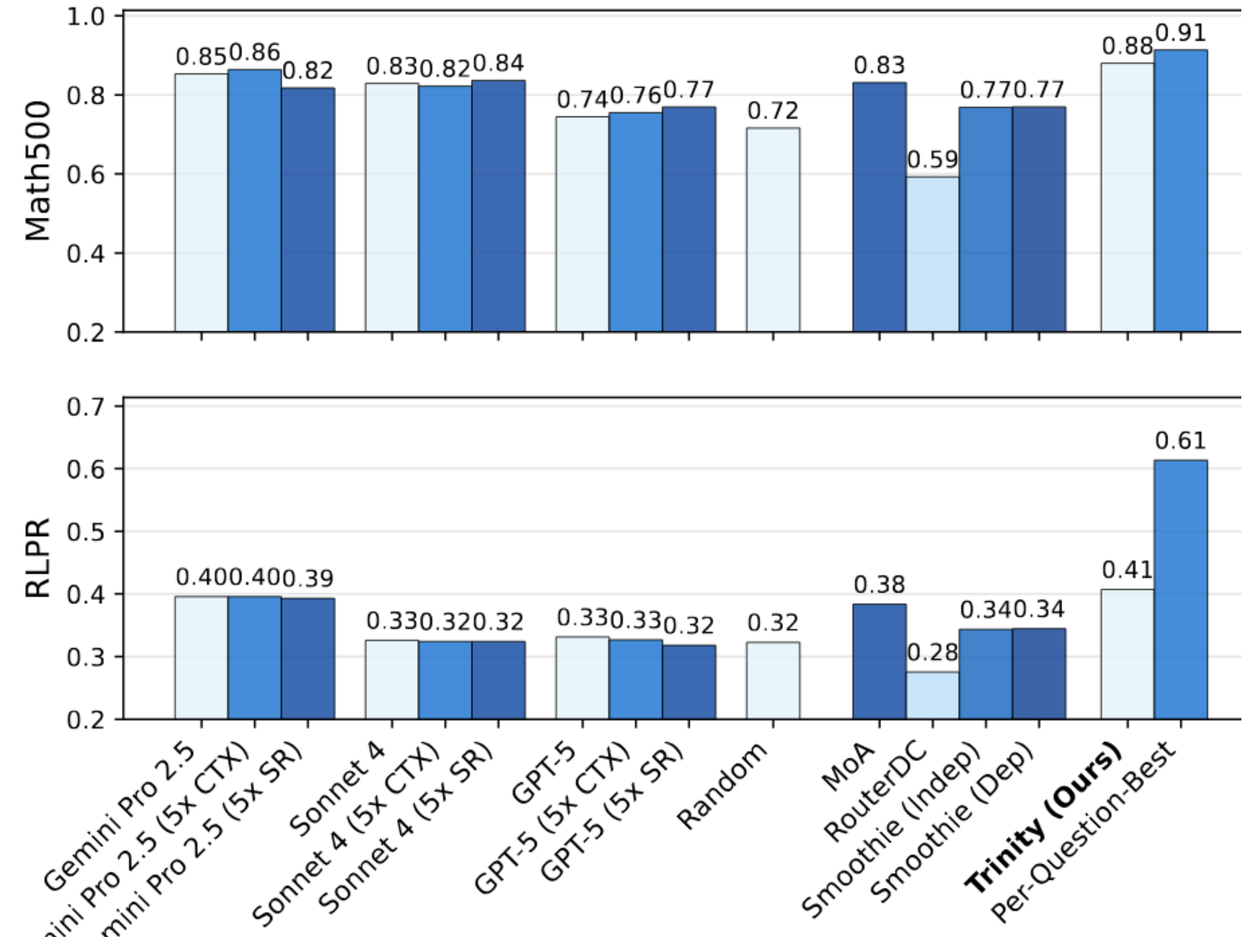
# Experiment setup

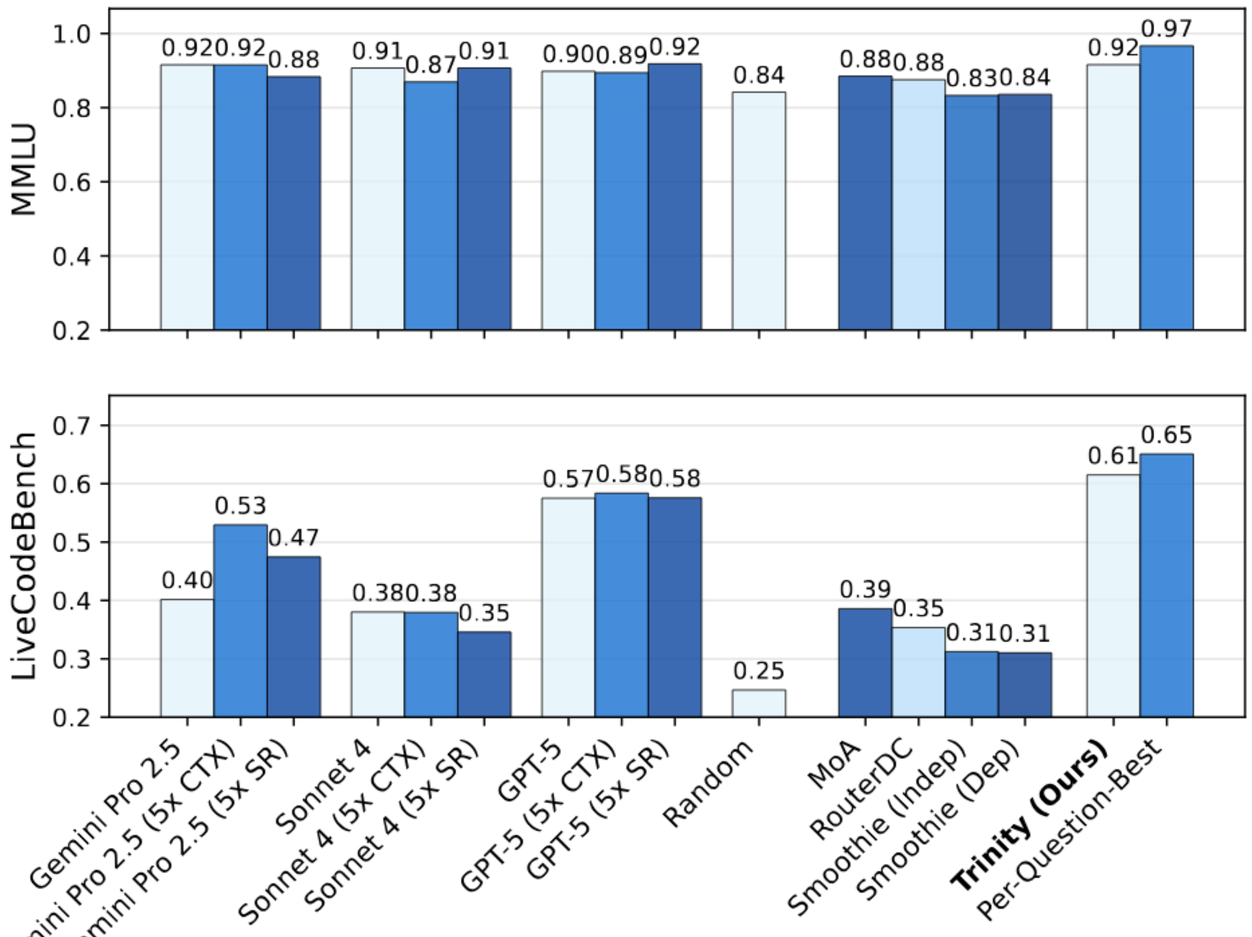
## Dataset, Baselines

- In-domain Datasets
  - MATH500: math QA dataset
  - MMLU: multi-domain MC dataset
  - RLPR: General reasoning QA dataset
  - LiveCodeBench: coding questions
- Out-domain dataset
  - AIME2025
  - BigCodeBench
  - MT-Bench
  - GPQA-D
- Generation Protocols
  - max_gen_token: 4096
  - max_turns: 5
  - Minimum reasoning effort
- Baselines
  - MasRouter
  - RouterDC
  - Smoothie
  - MoA
  - Random selection

# Main results

❓ Not sure if per-question-best include 5x variants with single model.




5x CTX: 5 times inference tokens
5x SR: self-reflection over 5 turns

TRINITY outperforms single- and multi-model baselines across four benchmarks.

# Ablation Study

Table 2: **Ablation study results.** We compare the performance on in-distribution tasks when we (1) remove the singular value fine-tuning in SLM; (2) remove the thinker-role selection (3) remove the tri-role selection; and (4) use the last instead of the penultimate token. TRINITY achieves the best overall performance. (5) remove agent selection but keep role selection

| Method | LiveCodeBench | MATH500 | MMLU | RLPR | Average |
|---|---|---|---|---|---|
| TRINITY | **61.46** | **88.00** | 91.56 | 40.72 | **70.44** |
| w/o Singular value fine-tuning | 55.68 | 85.85 | 90.10 | 39.77 | 67.85 |
| w/o Thinker-role selection | 57.80 | 86.20 | **92.75** | 38.00 | 68.69 |
| w/o Tri-role selection | 58.28 | 82.00 | 91.64 | 36.15 | 67.02 |
| w/ Last token | 50.85 | 87.00 | 82.19 | 38.60 | 64.66 |
| Claude-4-Sonnet only | 39.09 | 82.25 | 88.23 | 34.90 | 61.12 |
| Gemini Pro 2.5 only | 46.51 | 83.05 | 79.41 | **43.00** | 62.99 |
| GPT-5 only | 59.54 | 75.66 | 90.74 | 37.87 | 65.95 |
| | **0.58** (GPT-5 5 CTX) | 0.86 (Gemini Pro 2.5 5x CTX) | **0.92** (GPT-5 5 SR) | 0.40 (Gemini Pro 2.5 5x CTX) | |

❓ Without role prompt, is Trinity really doing effective routing other than selecting the best single model?

Assume single model baselines does not have agent role prompt

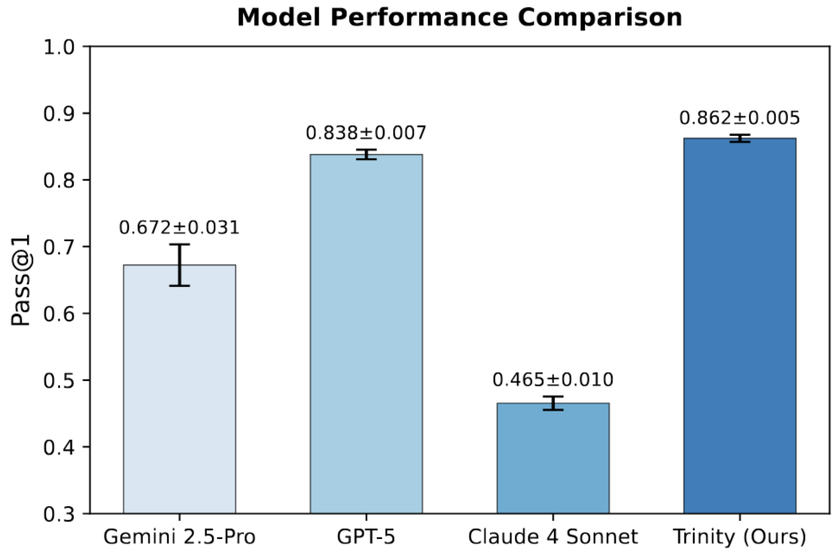
# Zero-shot transfer to unseen tasks

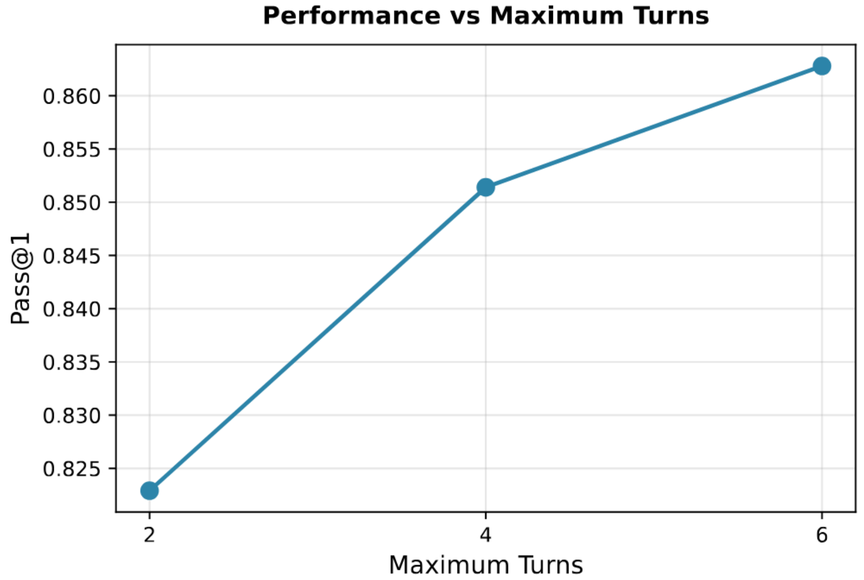
| Model | AIME | BigCodeBench | MT-Bench | GPQA-D | Average |
|---|---|---|---|---|---|
| Gemini Pro 2.5 | 46.67 | 35.10 | 9.37 | 75.25 | 52.34 |
| GPT-5 | 46.67 | 33.80 | 9.35 | 72.73 | 51.07 |
| Claude-4-Sonnet | 35.33 | **35.80** | 9.28 | 67.30 | 46.14 |
| Qwen3-32B (reasoning) | 23.33 | 20.90 | 8.99 | 59.09 | 34.44 |
| DeepSeek-R1-Qwen-32B | 30.00 | 24.30 | 8.43 | 51.01 | 35.10 |
| Qwen3-32B (direct) | 20.00 | 23.00 | 9.03 | 54.05 | 33.46 |
| Gemma-3-27B-IT | 20.00 | 20.30 | 8.76 | 33.33 | 21.38 |
| **TRINITY (Ours)** | **50.00** | **35.80** | **9.60** | **76.82** | **54.21** |

Trinity wins on 3 of 4 tasks outright and ties for first on BigCodeBench

# Full power on LiveCodeBench

- Remove the output-length constraint and use only the 3 closed source models, Trinity reaches a new high on LiveCodeBench V6.
- Increasing the max collaboration-turn budget from 2 to 6 improves pass@1 from 82.3% to 86.3%



# Compare with other optimization method

**Under same architecture (I assume)**

| Method | LiveCodeBench | MATH500 | MMLU | RLPR |
|---|---|---|---|---|
| REINFORCE | 0.253 | 0.459 | 0.500 | 0.266 |
| RS | 0.374 | 0.794 | 0.897 | 0.345 |
| SFT | 0.592 | 0.786 | 0.906 | 0.360 |
| sep-CMA-ES | **0.615** | **0.880** | **0.916** | **0.401** |

Policy gradient is not learning effectively. SFT is just single turn.

From proposition, sep-CMA-ES's improvement grows faster than RS with modest iteration number.

# Takeaways

- Penultimate token hidden state can act like input for routing
- Evolution algorithm (CMA-ES, sep-CMA-ES) is a tool for optimization problem without gradient
- Question: Is the performance boost purely from the routing algorithm or Trinity also benefits from the design of multi-agent collaboration?